

Question_3:

Code:

```
1 // There are n rooms and m directed teleports. Each teleporter can be used only
2 // once. Find the maximum number of days Teddy can travel from room 1 to room n
3 // and print the routes used for each day.
4 #include <iostream>
5 #include <cstring>
6 using namespace std;
7
8 #define MAXN 505
9 int capacity[MAXN][MAXN];
10 int adj[MAXN][MAXN];
11 int degreeCount[MAXN];
12 int parentNode[MAXN];
13 int visited[MAXN];
14
15 int bfs(int n, int s, int t)
16 {
17     memset(visited, 0, sizeof(visited));
18     int queue[MAXN], head = 0, tail = 0;
19     queue[tail++] = s;
20     visited[s] = 1;
21     parentNode[s] = -1;
22
23     while (head < tail)
24     {
25         int u = queue[head++];
26         if (u == t)
27             return 1;
28         for (int i = 0; i < degreeCount[u]; i++)
29         {
30             int v = adj[u][i];
31             if (!visited[v] && capacity[u][v] > 0)
32             {
33                 visited[v] = 1;
34                 parentNode[v] = u;
35                 queue[tail++] = v;
36             }
37         }
38     }
```

```
39     return 0;
40 }
41 int main()
42 {
43     int n, m;
44     cin >> n >> m;
45     for (int i = 0; i < m; i++)
46     {
47         int u, v;
48         cin >> u >> v;
49
50         capacity[u][v] = 1;
51         adj[u][degreeCount[u]++] = v;
52         adj[v][degreeCount[v]++] = u;
53     }
54     int maxDays = 0;
55     while (bfs(n, 1, n))
56     {
57         maxDays++;
58         int current = n;
59         while (current != 1)
60         {
61             int previous = parentNode[current];
62             capacity[previous][current]--;
63             capacity[current][previous]++;
64             current = previous;
65         }
66     }
67     cout << maxDays << endl;
68     for (int d = 0; d < maxDays; d++)
69     {
70         int path[MAXN], length = 0;
71         int current = 1;
72         path[length++] = current;
73         while (current != n)
```

```
74     {
75         for (int i = 0; i < degreeCount[current]; i++)
76         {
77             int v = adj[current][i];
78             if (capacity[v][current] > 0)
79             {
80                 capacity[v][current] = 0;
81                 current = v;
82                 path[length++] = current;
83                 break;
84             }
85         }
86     }
87     cout << length << endl;
88     for (int i = 0; i < length; i++)
89     {
90         if (i) cout << " ";
91         cout << path[i];
92     }
93     cout << endl;
94 }
95 return 0;
96 }
```

Output:

Test Case_1:

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
2
3
1 2 6
4
1 3 4 6
PS C:\Users\Prabin Yadav
```

Test Case_2:

```
6 7
1 2
2 3
3 6
4 4
4 5
5 6
5 6
1
4
1 2 3 6
PS C:\Users\Prabin Yadav\
```